

AXONA

Architecture

David A. Smith

Axona.net

An Axona Architecture Note v4.21.0 2026-07-13

*describes kernel 4.21.0 as deployed on testnet; written for human readers and AI
implementers alike*

A protocol is the contract between layers that don't trust each other.

What I cannot create, I do not understand.

RICHARD FEYNMAN · 1988

I How to Read This Document

This note is two documents sharing one text. Read as prose, it explains what Axona is and why each mechanism exists — every rule in this system was paid for by a specific failure, and the failures are named. Read as a specification, it contains everything needed to *re-construct* the protocol without access to the source: identifier derivations (§IV), the complete wire vocabulary with payload schemas (§VII, §VIII), the root-management decision table (§IX), the timing model (§XI), and the invariants any implementation must uphold (§XII). Section XVII is addressed directly to an implementer — human or AI — and orders the work.

Three conventions. *Normative* material (schemas, constants, invariants, algorithms) appears in tables and numbered rules; prose around it is *narrative* and explains intent. Feynman’s epigraph is the acceptance test: a reader who cannot re-derive the mechanism from this note has found a bug in the note, and should report it as such. And every constant is given with its value *and* its reason — a constant whose reason you don’t know is one you will mis-tune.

II The System in One Page

Axona is a peer-to-peer message substrate: publish/subscribe over a distributed hash table, with no servers in the data path. Every participant — a browser tab, a phone, a headless Node process — is a *peer* holding a keypair. Peers connect to each other directly over WebRTC data channels (browsers) or WebSockets (servers), discover each other through the DHT, and exchange *routed messages*: each message is addressed to a 264-bit identifier and forwarded, hop by hop, to the live peer whose own identifier is closest to the target by XOR distance.

Everything else is built from that one primitive:

- A **topic** is a 264-bit id derived from its name, owner, and write policy. Publishing routes the message toward the topic id; the peer where routing terminates is the topic’s **root** — emergent, never elected. The root timestamps (*stamps*) each message, giving the topic a total order, caches recent history, and fans deliveries out to subscribers.
- A **subscription** is a periodically renewed routed message toward the same topic id. The renewal is simultaneously the keepalive,

Companion notes: *Root Management (RootClaim state machine)* for the full election deep-dive; `INVARIANTS.md` in the kernel repo, mapping each invariant to its enforcing regression test; the *Kernel Refactor Analysis* for how the code reached its current shape; the *Soak-Test Framework* overview for how the live network is characterized.

the failure detector, the gap-recovery request, and the self-heal: if anything about the path or the root has changed, the renewal finds the new truth and re-attaches.

- **Durability** is replication among the peers XOR-closest to the topic (the *cohort*), plus explicit hand-off when a root departs gracefully.
- **Identity** is cryptographic and self-authenticating: a message is believed because it verifies against the author key it carries, never because of who relayed it. There is no certificate authority, no account database, and no server that can revoke a publisher.

One bridge server exists per network — for *signaling only*: it introduces new peers so they can open direct connections. It is bootstrap infrastructure, not a data-path relay; two peers with established mesh connections continue exchanging messages if every bridge is down, and new connections can even be signaled across the existing mesh.

The design bets on one asymmetry, everywhere: *wrong claims that self-correct are cheaper than coordination that prevents them*. Roots are claimed optimistically and reconciled by evidence; publishes are retried idempotently rather than acknowledged; caches converge by anti-entropy rather than consensus. The protocol has no leader, no quorum, and no distributed lock anywhere.

Proven live 2026-07-12: a fresh authenticated data channel formed between two peers with the bridge process dead, signaling relayed over the mesh (19/19, bulk 100%).

III The Stack

Layer	Repo / package	Role
Kernel	@axona/protocol	identity, routing, transports, pub/sub
Bridge	axona-bridge	WebSocket signaling + TURN minting; embedded peer
Relay	axona-relay	headless supernode (hosting, metrics, ops CLI)
Peer app	axona-peer	reference browser application (axona.net)
Simulator	dht-sim	1K-50K-node simulation over the vendored kernel

The kernel is the protocol; everything else consumes it. Its pub/sub internals were consolidated in the 4.19–4.21 refactor into single-concern modules behind one façade — the module map doubles as the concept map of this document:

Module (src/pubsub/)	Lines	Owns
AxonaManager.js	~500	façade: state, public API, routing egress
wireHandlers.js	~700	routed handlers + axon-tree mechanics
repairPlane.js	~400	the tick scheduler, retries, replication, departure
rootElection.js	~290	beacons, root hints, self-verification, liveness
rootClaim.js	~240	the isRoot state machine (§IX)
constants.js	~170	every tunable + wire types (§XI)
topicStore.js	~150	cache, tombstones, exactly-once app delivery
envelope.js / post.js / kill.js	—	signing, canonical form, ids

Outside pub/sub, the kernel provides `src/dht/AxonaPeer.js` (the peer object: lifecycle, the local node, routing entry points, the public

API of §XIV), `src/transport/` (web, node, sim transports; handshake; mesh management), `src/identity/`, and `src/utls/hexid.js` (the keyspace profile: 264-bit production ids, reducible for simulation).

IV Identity and Addressing

Everything in Axona lives in one keyspace: **264-bit identifiers**, written on the wire as 66 lowercase hex characters, handled internally as `BigInt`. An id is always `regionByte` $\parallel$ 256-bit `body`. The high byte is an S2 geographic cell (a coarse region code); the remaining 256 bits are cryptographic material. Distance between ids is XOR, compared as unsigned integers; the region byte's position makes same-region ids numerically close, so proximity in the keyspace loosely tracks proximity on the planet.

Node identity (where routing can reach you)

`createNodeIdentity({lat, lng})` generates an Ed25519 keypair and derives `nodeId = regionByte(lat,lng)  $\parallel$  SHA-256(pubkey)`. The node key authenticates *transport*: channel establishment, handshakes, routing — never content. Node identity is typically ephemeral; a peer that restarts with a fresh key is simply a new node.

Author identity (who said it)

`createAuthorIdentity()` generates a separate, location-free Ed25519 keypair; the `authorId` is the public key. Author keys sign *content* and are usually persistent (the browser app stores them in IndexedDB, non-extractable). The separation is deliberate and load-bearing: the network learns *where* a node is from its node id but only *who* authored a message from its envelope — a signed envelope discloses the author, never the author's location, and no protocol change may add a node-id or region to the envelope.

This rule has been defended twice against convenience patches. Location display, where an application wants it, is app-level and opt-in.

Topics (structured addresses with policy folded in)

A topic is addressed by a *descriptor* `{region, owner, name, write}`: `region` a region name or code; `owner` an `authorId` or null; `name` a UTF-8 string; `write` either `'open'` (anyone publishes) or `'owner'` (only the owner's key). The id is:

```
topicId = regionByte
          || SHA-256( canonical({ owner, name, write }) )
```

with the region byte resolved by rule: an explicit `region` wins; otherwise, if `owner` is present, the region is *derived from the owner key* (`keyDerivedRegion`: hash the `authorId` into the list of populated land

regions) so an author's topics are discoverable from the `authorId` alone; otherwise the derivation throws (`TOPIC_REGION_REQUIRED`). Because the write policy is folded into the hash, a topic's policy is not mutable state — a root recomputes the id from the signed descriptor and enforces `write: 'owner'` statelessly.

Messages (content-addressed, author-signed)

Every published message travels as an **envelope**:

```
{ msgId:  hex64,           // content address, see below
  seq:    int,             // per-publisher monotonic
  ts:     ms-epoch,        // publisher clock
  topic:  { region, owner, name, write }, // the signed descriptor
  message: <any JSON value>,
  signature?: 'ed25519:' + hex128,
  signerPubkey?: hex64 } // the author key (absent = anonymous)
```

- `msgId = SHA-256(canonical({publisher, message}))` where `publisher` is `signerPubkey` or null — a stable content address of (author, message), independent of time, topic, or signature. Identical re-publishes dedup network-wide by construction; a publisher wanting distinct ids adds a nonce to `message`.
- The signature covers the domain-tagged core `canonical({d, seq, ts, topic, message})` with `d` the envelope domain tag — domain separation means an envelope signature can never be replayed as a kill signature or a channel-binding value; signing the resolved descriptor binds the message to the exact topic *and policy*.
- `canonical()` is a total, JSON-valid canonical encoding (sorted keys, RFC-8785-style), shared by every signing and hashing site in the system.
- Freshness: receivers enforce a bound on `ts` skew and per-publisher `seq` monotonicity at live ingress (anti-replay); replayed history is exempt from the `ts` bound but capped by a future-tolerance rule (§VIII).

A **kill** (retraction) is its own signed object naming (`topicId`, `msgId`), domain-tagged, and valid only when signed by *the same author key* that signed the target message — creator-only retraction, enforced at every root, with a provisional path when the kill races ahead of its target (§VIII).

V Routing

Each peer maintains a **synaptome**: a bounded table (target ~ 50 entries) of live, authenticated, directly-connected neighbours, mixing keyspace-near peers (completeness around one's own id) with long-range contacts (reach). Entries are first-party only — gossip feeds a *candidate pool*, and a candidate becomes a synapse only after a verified direct connection, which is the eclipse defense: no one can write into another peer's routing table.

Two search primitives share the table:

Greedy routing (*routeMessage*) forward to the neighbour strictly closest to the target; the peer where no neighbour improves on self is the *terminal*. One-pass, no backtracking, hop-budgeted. Fast and usually right; on a sparse or divergent mesh it can strand in a local minimum — every convergence mechanism in §IX exists to correct exactly this.

Iterative lookup (*findKClosest* / *lookup*) the classic α -parallel iterative search, querying successively closer peers for their neighbours. Slower (multiple round trips) but global: it escapes local minima and crosses regions. The kernel *never* blocks a send on it — lookups warm hints in the background (§IX).

Two standing rules. **The bridge is never a data hop**: routing skips the bridge id even when it is numerically closest — signaling infrastructure must not become a root, a relay, or a convergence anchor.

Region is a placement hint, not a wall: the region byte clusters same-region ids, but a lookup crosses regions freely; treating the prefix as a routing wall caused a cross-region split-brain (fixed 4.17.1) and is now an explicit anti-pattern.

Departed peers are marked dead immediately on channel loss (eviction, graceful-leave notify) and skipped by routing; reachability to a *newcomer* lives in its neighbours' tables and is healed by inbound traffic — which is why cold publishers deliberately re-send (§XI) rather than waiting to be found.

VI Transports and the Bridge

The kernel is transport-agnostic behind one interface; three implementations ship. `Transport.web()` — WebRTC data channels between peers, WebSocket to the bridge for signaling; the browser production path. `Transport.node()` — WebSocket server-to-server; relays and bridges. `Transport.sim()` — in-process, for the simulator and tests.

Authentication: axona/5

Every channel is mutually authenticated before any Axona frame flows. The handshake proves possession of the node key via a signed, domain-tagged challenge bound to the channel: for WebRTC, the channel-binding value folds both sides' *DTLS certificate fingerprints* (a MITM terminating DTLS changes the fingerprints and the signature fails); for WebSockets, the binding uses the connection's nonces. A peer's claimed nodeId must equal `regionByte || SHA-256(pubkey)` for the proven key — identity is verified, not asserted.

Version discipline

Two version axes ride the hello frames. `WIRE_VERSION` (currently 4.0) partitions hermetically on its major — wire-3.x and wire-4.x reject each other at both the bridge gate and the peer-peer handshake, because 4.x changed convergence and replay semantics. `KERNEL_VERSION` (currently 4.21.0) is informative and surfaced at every runtime surface (healthz, welcome, UI) — an observability rule, not a gate; wire-compatible kernels interoperate freely, and deploys are staggered on exactly that property.

The bridge, and life without it

The bridge accepts WebSocket connections, runs the version gate and the authenticated hello, mints short-lived TURN credentials, and *introduces* peers: a joining peer receives a set of live peers to open WebRTC connections to, and the bridge ferries the SDP/ICE signaling for those first connections. The bridge embeds an ordinary kernel peer (it hosts keySPACE, participates in routing for signaling purposes) but is excluded from data-path roles by every selection rule.

Join flow: connect WSS → version gate → authenticated hello → welcome (peer list, TURN) → open direct channels → synaptome admission per connection → the peer is routable. `connect()` wraps the whole sequence in one call. After bootstrap the bridge is optional: *mesh-relayed signaling* routes SDP/ICE across existing data channels, so new direct connections form bridge-free, and existing connections never depended on it.

Bridges self-advertise on the public topic `axona:bridge-directory` (each bridge `host()`s and republishes it); clients collect the directory at launch, rank, and fail over. Multiple bridges federate by uplinking to each other as ordinary peers, so their peer populations form one mesh.

Judging clients fairly

A bridge judges client liveness only over time it was actually listening: an event-loop stall on the bridge re-arms client grace and suspends idle sweeps until the taint ages out (invariant I-5). A client must never be dropped for the server’s own pause.

VII The Wire Protocol

All frames are JSON. Above the transport layer there are three frame families: the handshake/hello family (§VI), direct notifications (`direct_*`, point-to-point over an existing channel), and *routed messages* — the workhorse. A routed message carries `{target, type, payload, fromId, hopBudget}` and is forwarded greedily toward `target`; the handler registered for `type` runs at the terminal (or at a via waypoint — see below).

Pub/sub message types (normative; payload schemas in §VIII):

Type	Routed toward	Meaning
<code>pubsub:sub</code>	topic (or via)	subscribe / renew; carries since, hw
<code>pubsub:unsub</code>	topic (or via)	explicit unsubscribe
<code>pubsub:pub</code>	topic (or via)	publish; unstamped envelope
<code>pubsub:deliver</code>	subscriber	stamped messages (and del markers)
<code>pubsub:adopt</code>	child	delegate: become child relay, take batch
<code>pubsub:pullup</code>	holder	“you are ahead of me — replay up”
<code>pubsub:replayup</code>	parent	stamped cache delta, upward
<code>pubsub:handoff</code>	heir	departing root pushes history
<code>pubsub:replicate</code>	cohort member	root pushes cache+tombstones
<code>pubsub:rootbeacon</code>	neighbours	soft-state root advertisement
<code>pubsub:kill</code>	topic (or via)	signed retraction
<code>pubsub:pull</code>	topic (or via)	one-shot read; <code>pullresp</code> returns
<code>pubsub:metricson</code>	topic (or via)	demand-driven metrics lease
<code>pubsub:touch</code>	topic	reserved no-op (wire compat)
<code>pubsub:unpub</code>	—	reserved string, no handler

Via waypoints. Topic-addressed payloads may carry `via: [hex,...]` (capped at 8), an ordered waypoint list: the message routes toward `via[0]` first. At the terminal for a waypoint that is not the local node, the waypoint is *popped* and the message re-sent toward the next (finally the bare topic id). This is the oldest self-healing rule in the system — *a dead waypoint always falls through to the topic id and re-seats* — and it is load-bearing: pinned subscriptions to departed relays heal by exactly this mechanism, and any analysis that forgets it will wrongly predict deadlocks.

The topic decision. Every topic-addressed handler first classifies: *handle* (I am `via[0]` and hold the role, or I am the bare-topic terminal and region-permitted), *reroute* (dead or foreign waypoint at my terminal — pop and continue), *forward* (not terminal), or *reject* (terminal but the region lock forbids rooting here; enabled only when `configureRegionLock({enforce:true})`, which is OFF pre-critical-mass).

An external review did precisely this in July 2026; the trace is in the Root-Management review response.

VIII Pub/Sub: The Axon Tree

Pub/sub is *routing-only*: every interaction is a routed message; there are no direct pub/sub connections, no root sets, no K-closest fan-out at publish time. One rule generates the whole design: *the peer where a topic-addressed message terminates acts for the topic*.

Roles and per-topic state

A peer holding any responsibility for a topic keeps a `role`: `{topicId, isRoot, subscribers (map subHex  $\rightarrow$  {since, lastRenewed}), children (set of child-relay ids), cache (ascending by stamp), tombstones, replicas, seq, lastTs, metricsOn}`. `isRoot` is the root-claim state machine's state and changes *only* through it (§IX).

Subscribe: attach, renew, re-home

`sub(topic, {since})` sends `pubsub:sub {topicId, subscriberId, since, hw, via, latest?}`. `since` is the replay floor: a timestamp, 0 (“`since:'all'`” — full retained history), or now (live tail); `latest` additionally folds the newest retained entry into the first delivery regardless of age. `hw` advertises the sender's own cache high-water — the durability hook: a root that is *behind* a reattaching subscriber issues `pullup` and adopts the newer history without re-stamping.

The serving relay *seats* the subscriber, replays the cache delta above `since` (chunked at 96 KB per deliver), replays all live tombstones (unconditionally — a since-gate cannot express “you missed an old deletion”), and answers with a deliver whose `from` field *pins* the subscriber: subsequent renewals go *via* the pin. Renewal is adaptive: 5 s while unpinned or freshly re-pinned, backing off $\times 1.5$ per stable renewal to a 60 s ceiling; any re-pin or upstream death snaps it back to fast. Subscribers are evicted after 180 s without renewal.

The tree: widen before deepen

A relay over capacity (20 direct) *promotes* one in-region leaf to a child relay and hands it a batch of up to 8 others (`pubsub:adopt`); the child subscribes up toward the parent and re-fans deliveries down to its own subscribers exactly once. Only when every direct is already a child does the tree deepen (the newcomer is delegated to the XOR-closest child). Depth grows like $\log_{20}(\text{subscribers})$.

Publish: stamped at the root, confirmed by observation

`pub(topic, message, {signWith})` builds the signed envelope and sends `pubsub:pub {topicId, via, json}` toward the topic (via the current root hint when warm). At the root: verify the signature (B-4), check freshness (C-2), recompute the `topicId` from the signed descriptor and reject a mismatch, enforce `write: 'owner'`, dedup by `msgId` and tombstone — then `stamp: publishTs = max(lastTs+1, now)` (strictly monotonic; the total order) and `seq = ++role.seq` (dense per-topic counter; a gap in delivered `seq` tells a subscriber it missed something). The stamped message is cached, fanned to subscribers, delivered locally, and eagerly replicated to the cohort.

There is no publish acknowledgment, by design: a PUB carries no return address (publisher-location privacy). Confirmation is *observation* — the publisher stops retrying when it sees its own `msgId` arrive by any path (own delivery, own root ingest, cohort echo). Until then the pending-publish machinery re-sends the same envelope toward the current root hint each tick, bounded by 30s and 6 tries; idempotent throughout because roots dedup by `msgId`. Cold publishers (fewer than 8 neighbours) front-load extra re-sends (§XI) — each send both integrates the newcomer and gets a fresh shot at the true root.

Durability: cohort, replay-up, handoff

Three migration paths keep history alive, and every one carries tombstones alongside bodies (invariant I-8 — migrating a cache without its deletions resurrects killed messages):

1. **Cohort replication** — each root pushes its full cache+tombstones to its 2 topic-closest reachable non-bridge peers (`pubsub:replicate`), every tick and eagerly on every stamp and kill. Receivers become *backups*: warm, verifying copies that also subscribe toward the topic like ordinary child relays — so on root churn, *election is just subscription*: the closest backup's renewal terminates at itself and it promotes with the history already in hand; the rest re-home under it. Idempotent full-state push doubles as anti-entropy: co-holding roots converge to the union.
2. **Replay-up** — a root behind a cache-bearing attacher (**hw** says so) pulls the difference up (`pullup/replayup`) and adopts it *without re-stamping* (a timestamp once assigned is never reassigned), advancing `lastTs/seq` so new stamps continue above.
3. **Graceful handoff** — §X.

Replayed stamps more than 5 minutes in the future are dropped (the bad-clock rule). Caches are bounded (1024 messages / 16 MB per role) and expire at 24 h from stamp.

Kill: creator-only retraction

`kill(topic, msgId)` routes the signed kill like a publish (same hint, same pending-retry). The root verifies the kill’s own signature, then authorship: if the target is cached, `signerPubkey` must match the target’s author *now*; if not cached (kill raced its target), the kill is accepted *provisionally* — the tombstone records the signer, and when the target arrives, a mismatch *revokes* the tombstone and accepts the message. Kills are stamped like publishes (they occupy a `seq` slot), fanned as `del` markers, replicated eagerly to the cohort, and replayed on every renewal while live. A subscriber that never received the body gets no retraction callback (nothing to retract); the tombstone still converges.

Pull, metrics, host

`pull(topic, msgId?)` is a one-shot read answered by the *first replica* the routed request reaches (by-`msgId` is exact by content address; pull-latest is deliberately “recent, not necessarily newest” — hot read paths spread across the cohort instead of hammering the root). `metrics` are demand-driven: a `metricson` routed like a SUB arms a renewable 70s lease at the root, which then publishes signed snapshots `{topic, ts, by, current_count, seq, subscribers, bytes}` to the derived topic `metricTopic(T)` every 20s while the lease is fresh — no lease, no load; leases survive root promotion via path flags. `host(topic?)` makes infrastructure carry without consuming: a hosted topic is renewed like a subscription (so the node can root it and its cache is durable) but nothing is delivered to the application; no-arg `host()` volunteers for the node’s whole keyspace neighbourhood (retaining any role it wins as terminal) — the relay fleet’s default mode.

IX Root Management

The root is emergent — the live peer XOR-closest to the topic id, discovered by routing — so root management is a *convergence* protocol, not an election: wrong claims must yield to right ones, quickly, without flapping, and never to a node that is farther, dead, or departing. Since kernel 4.20.0 every `isRoot` transition passes through one state machine (`rootClaim.js`) with one structured log line per flip.

The companion note *Root Management — the RootClaim state machine* gives the full treatment with worked examples; this section is the normative summary.

Transitions

Transition	why-code	Trigger
become	sub/pub/kill/metricson-terminal	topic message terminated here, no role
become	handoff-heir	departing root handed us history
promote	terminal-promote	non-root relay became bare-topic terminal
demote	defer-terminal	stranded traffic deferred to beaconed root
demote	beacon-closer	beacon proved strictly-closer live root
demote	verify-closer	self-verification found closer terminus
demote	handoff-better-heir	post-handoff closer live root ($\neq$ leaver)
claim	reachable-fallback	deferral unconfirmed 6s; self closest reachable
adopt	adopted-child	ADOPT made us a non-root child

Every demotion does three things atomically as policy: flip `isRoot`, pin upstream to the winner, and send a confirming subscribe (the winner must *adopt* us or our subtree starves).

The defer gate (never yield to farther, ghost, or leaver)

Before any claim, `liveCloserRoot(topic)` consults the cached root beacon for the topic. Rules, in order: no fresh beacon $\rightarrow$ claim; beacon names self or a node *not strictly closer* $\rightarrow$ claim (the cardinal rule — also the security property: a lying beacon cannot divert traffic outward); otherwise liveness evidence is required, strongest first: a **verified** pointer (network-confirmed by iterative lookup, TTL 90s), a channel-verified direct neighbour (opens *instantly* on churn — the dead root’s channel drop is the gate), or — only for the loose mode used by PUB/KILL — a beacon heard within 30s (a live root re-beacons every 20s; silence is death). SUB, METRICSON, promotion, and post-handoff use the strict mode: seating infrastructure demands channel or network evidence.

The evidence planes

Beacons: every root advertises `{root, topics[]}` to its 6 XOR-closest neighbours, forwarded 2 layers (basin ≈ 42), every 20s and immediately on becoming root; pointer TTL 50s; receivers apply verify-don’t-trust (accept only if at least as close as the best locally-known node). A wrong claimant hearing a strictly-closer beacon demotes at once — which also silences its own poisoning beacons. **Self-verification:** beacons only reach the basin, so every root re-checks its own claim with the same iterative lookup subscribers use — at 6s after forming (the fresh-topic race window), then every 45s, batched 3 per tick, never awaited inside the tick. Finding a strictly-closer live node seeds a **verified** pointer and demotes. **Death sweeps:** when a peer’s channel closes or it announces leave, every beacon naming it *and every upstream pin on it* is purged, with the pinned subscription’s renewal clock reset — re-homing is next-tick, and the reachable-root

fallback re-arms.

The fallback asymmetry

A subscriber unpinned for 6s despite renewals (its hint names a closer-but-unreachable node) claims the root itself if it is closest among *reachable* peers: a reachable root beats a closer unconfirmed one.

Claiming wrongly is cheap — beacons or verification demote it within one period; deferring wrongly strands the topic forever. When the mesh is bare (no non-bridge neighbours), a self-SUB terminal claims nothing — “terminal at self” on an unmeshed node is isolation, not closeness; the seat is held and the fast renewal re-runs the decision once meshed. Publish-side is deliberately not gated: a genuinely solo node still roots its own publish.

X Lifecycle: Join and Leave

Join: `connect({bridge, identity?})` performs the bridge bootstrap of §VI, resolves the directory, and returns a started peer. Reachability ripens over the first seconds as inbound traffic writes the newcomer into neighbours’ tables; the cold publisher burst and the fast renewal floor are shaped around exactly this window.

Leave (`leave({timeoutMs})`) is a contract with three promises, each bought by an incident:

1. *Drain on evidence:* poll until the pending publish/kill maps are empty — a confirmed publish departs instantly, an unconfirmed one gets the caller’s full window (never a hidden cap). Internal waits are ref’d timers: Node must not exit mid-leave.
2. *Hand off every rooted topic:* for each rooted, cache-bearing topic, resolve an heir — local K-closest first, falling back to the *iterative* lookup when the leaver’s table is thin (the fresh burst-publisher case) — with 8 resolutions in flight, all raced against the timeout; push `pubsub:handoff {topicId, msgs, dels, from}.from` names the leaver so the heir purges the leaver’s ghost beacon and never defers its new claim back to it.
3. *Then silence:* notify peers in parallel (they run the death sweep immediately), stop the tick, clear all retry state. A peer that has left sends nothing — and the network stops listening for it.

Abrupt death is the same story without the courtesy: channels drop, neighbours sweep beacons and pins instantly, the warm cohort holds the history, and election-by-subscription seats the successor.

XI The Repair Plane and Timing Model

One scheduler (`refreshTick`, every 5 s) drives all periodic repair: subscription renewals (adaptive), hosted re-announces, backup renewals (every tick — infrastructure never backs off), the pending publish/kill retry, cohort replication, metrics leases and snapshots, subscriber eviction and cache/tombstone expiry, role teardown, beacon emission, and the self-verification batch. The only timers outside the tick are the cold-publish burst and the one-shot first-publish re-send — all tracked and cleared on stop/leave.

Constant	Value	Reason
tick	5 s	must be $\leq$ the renewal floor
RENEW_FAST_MS / RENEW_MS	5 s / 60 s	orphan window vs. steady traffic ($\times 1.5$ backoff)
DROP_MS	180 s	subscriber eviction ($\geq 3\times$ ceiling)
ROOT_CLAIM_MS	6 s	unconfirmed-deferral window (≥ 2 ticks)
BEACON_MS / TTL	20 s / 50 s	advert cadence; $1.5\times$ = corpse-freshness cut
BEACON_FANOUT / layers	6 / 2	basin ≈ 42 nodes
verify first / steady / batch	6 s / 45 s / 3	fresh-topic race; steady re-check; no lookup storm
ROOT_REPLICAS	2	cohort size (singleton-root durability)
pending TTL / tries	30 s / 6	bounded retry; loss ⁷ negligible at 30%
cold burst	5×200 ms + 5×400 ms	integrate-and-retry while the table warms
first-publish re-send	200 ms	catch a tree formed microseconds before
MAX_DIRECT / batch	20 / 8	delegation threshold / handoff batch
cache	1024 msgs / 16 MB / 24 h	bounded hold, keyed on root stamp
metrics lease / cadence	70 s / 20 s	demand-driven; self-expiring
future tolerance	5 min	bad-clock rule for replayed stamps

Convergence intuition: a wrong claim inside the beacon basin corrects within ≤ 20 s; outside it, ≤ 6 s fresh / ≤ 45 s steady; a deferral to an unreachable node resolves in ~ 6 s; a dead root’s influence ends at channel-close for neighbours and ≤ 30 s otherwise; a subscriber whose neighbour upstream dies re-homes next tick; a backed-off subscriber heals no later than its next renewal via the waypoint-pop rule.

XII Invariants

Normative. Each is enforced by a named regression test in the kernel repo (`INVARIANTS.md` maps them); an implementation that violates one is wrong even if it benchmarks well.

- I-1** A topic has exactly one root; wrong claims converge without flapping.
- I-2** Never defer a root claim to a farther node, a ghost, or the node handing off.
- I-3** A recovery path never waits unboundedly on — or dies on — the failure it handles.
- I-4** A peer that has left is silent, and its rooted history departs before it does.

- I-5** A client is never judged by time the server wasn't listening.
- I-6** Observability surfaces exist or fail loudly — never silently zero.
- I-7** Fixes must hold for 100%-transient peers: no mechanism may privilege stable nodes to mask churn.
- I-8** Migrated cache never resurrects a killed message: every migration path carries tombstones, applied first.
- I-9** Publish confirmation is observation, not acknowledgment; nothing may add an ack channel that discloses the publisher's location.

XIII Robustness: What Fails, and What Happens

A subscriber's relay dies. Neighbour case: death sweep drops the pin, renewal snaps fast, re-home next tick. Otherwise: the next renewal routes toward the corpse, pops at the live terminal, re-seats at the true root, and the deliver re-pins — worst case one renewal interval.

The root dies abruptly. Its beacons are swept by neighbours; the warm cohort already holds cache+tombstones; the closest backup's next renewal terminates at itself and it promotes, gap-free; the rest re-home under it. Singleton topics survive because replication is proactive, not reactive.

The root leaves gracefully. Handoff pushes history to heirs (parallel, timeout-raced, thin-table lookup fallback); heirs never defer back to the leaver.

A publish strands. The greedy walk hit a local minimum; the pending retry re-sends toward the refreshed hint each tick (bounded); cold publishers burst. The retrying is idempotent end-to-end.

Two roots exist transiently. Near-miss terminals can claim while evidence is thin — by design. Beacons reconcile within the basin; self-verification reconciles across it; cohort anti-entropy keeps the data unified while the claims converge; a since-all rejoin reads the union.

The network partitions. Each side converges internally (roots, cohorts). On heal, beacons and verification collapse duplicate roots; anti-entropy unions the caches; per-publisher `seq` lets readers order interleaved histories.

The bridge restarts. Existing mesh connections are unaffected (the bridge is signaling-only). Reconnecting peers ride through transient 502s — the reconnect path survives the very errors reconnection

produces (I-3) — and re-establish. Nothing about topic state lives on the bridge.

A peer lies. A forged envelope fails B-4 verification at the root; a forged kill fails authorship and is revoked even if provisionally recorded; a lying beacon cannot point farther than the receiver’s best-known node; gossip cannot write routing tables; a claimed `nodeId` must hash from the proven key.

Clocks are wrong. Stamps are root-monotonic regardless of wall clocks; replayed stamps too far ahead are dropped; freshness gates use bounded skew.

XIV The Application API

The kernel’s public surface on `AxonaPeer` (browser and Node identical): lifecycle — `connect()` / `join()`, `start()`, `leave({timeoutMs})`, `stop()`, `ready()`; pub/sub — `pub(topic, message, {signWith})`, `sub(topic, {since, replayLatest})`, `unsub`, `kill(topic, msgId)`, `pull(topic, msgId?)`, `metrics(topic)`, `host(topic?)`, `unhost`; identity — `createNodeIdentity`, `createAuthorIdentity` (module exports); direct messaging — `send/notify/onMessage`; introspection — `health()` (roles, hosting, mesh truth), `peers()`, `rootedTopics()`, `onPeerJoin/onPeerLeave`, `onLog/onError`, `onUpgradeRequired`; persistence — `snapshot()/fromSnapshot()` plus the `PersistenceAdapter` (IndexedDB, file, memory). Topic arguments are descriptors (§IV); `signWith` takes an author identity or the `ANONYMOUS` sentinel. Applications render message bodies through `std/message` and move bulk bytes through `std/chunk` (10 KB chunks under the 15 KB reliable floor).

XV Security Model

Self-authentication end to end: content believed only against the author key it carries (B-4 at every root ingress and every stamped re-ingest); transport believed only against the node key proven on the channel (axona/5, DTLS-fingerprint binding on WebRTC); freshness and per-publisher monotonic `seq` against replay (C-2); domain-tagged signatures everywhere (no cross-protocol replay); total canonical encoding shared by signer and verifier (C-1); inbound size and count caps on every attacker-controllable field (D-1: 256 KB hard publish ceiling, 15 KB reliable floor, via ≤ 8 , beacon topics ≤ 256); first-party-only routing tables against eclipse (B-3); verify-don’t-trust on every hint (beacons, pull answers, replayed history). Address grinding is answered by memory-hard proof-of-work on a *separate puzzle*

$H(\text{domain} \parallel \text{role} \parallel \text{pubkey} \parallel \text{nonce})$ — never on the address itself, which would skew the keyspace (calibration complete; activation gated). The standing privacy rule bears repeating: the protocol learns *who* from envelopes and *where* from node ids, and no layer may join the two.

XVI *Production Deployment*

Two isolated networks, partitioned hermetically by wire major.

Production: bridges east (`bridge.axona.net`, NYC) and west (`bridge-west.axona.net`, SFO), Dockerized, federated by uplink; a nine-relay backbone (three droplets $\times$ three relays: SFO/NYC/TOR) running keyspace hosting; the reference app at `axona.net`. **Testnet:** the staging line (`testnet.axona.net` bridge + apps) — every kernel release soaks here first; prod promotion is a deliberate, evidence-gated step (full suite, external axonSpec gate, settled-mesh soak acceptance, and the standing runbook). Every component surfaces its `KERNEL_VERSION` at a runtime-inspectable surface; deploys are staggered and judged only after the mesh settle window, never by one-shot fresh clients.

XVII *Reconstruction Guide*

To rebuild a conformant peer from this note alone, implement in this order, testing each layer before the next:

1. **Canonical form and ids** (§IV): total canonical JSON; SHA-256; Ed25519; the 264-bit id discipline (region byte, $\text{hex} \leftrightarrow \text{BigInt}$); `deriveTopicId` with its region-resolution rule; `msgId`. Test: same inputs, byte-equal outputs, cross-implementation.
2. **Envelope and kill** (§IV): build / sign / verify with domain tags; freshness and seq rules. Test: a signature from one implementation verifies in the other; domain confusion fails.
3. **Routing** (§V): synaptome with first-party admission; greedy `routeMessage` with terminal detection and hop budget; iterative `findKClosest`; dead-peer skip; bridge exclusion. Test: terminal correctness on random topologies; lookup escapes a constructed local minimum.
4. **Transport and handshake** (§VI): any reliable channel + the axona/5 mutual auth with channel binding; the wire-major gate. Sim transport first; real transports are substitutable.
5. **The routed pub/sub core** (§VII, §VIII): the topic decision (handle / reroute / forward / reject) with waypoint popping; roles;

SUB seating + replay + pinning; PUB verify-stamp-cache-fanout; DELIVER re-fan and re-pin. Test: N subscribers, one root, 100% delivery, total order by stamp.

6. **The root-claim state machine** (§IX): the transitions and the defer gate with its three evidence tiers, as one module with one transition function; then the evidence planes (beacons with verify-don't-trust, self-verification, death sweeps). Test: the decision table directly — farther beacons never defer; ghosts never defer under strict mode; every flip logs once.
7. **The repair plane** (§XI): one tick driving renewals (adaptive), pending retries, replication, eviction, expiry, beacon/verify cadence; the constants table verbatim; complete teardown on stop.
8. **Durability + lifecycle** (§VIII, §X): cohort replication with union anti-entropy; replay-up; kills with provisional authorship; leave with evidence-drain, parallel handoff, and silence.
9. **Prove the invariants** (§XII): write the nine as executable tests before tuning anything; then characterize on a live mesh — healed delivery after a renewal cycle is the headline number, single runs are noise ($\text{REPS} \geq 5$), and no verdict comes from a seconds-old client.

Two traps this document exists to prevent: forgetting the waypoint-pop rule (you will wrongly predict deadlocks and “fix” them into churn storms), and adding a ninth root-claim site instead of a guard in the one decision table (you will recreate the patch-interaction bug class this architecture was refactored to kill).

XVIII Appendix: The Life of a Message

Publisher P (browser, us-east), subscribers $S_{1..12}$, fresh topic $T = \{\text{region: useast, owner: null, name: 'lobby', write: 'open'}\}$.

1. S_1 subscribes: SUB routes toward T , terminates at R (the closest live peer) $\rightarrow$ `become('sub-terminal')` — R is now the root, beacons immediately, self-verifies at 6s. S_1 is seated, pinned by the deliver-from, renews via R .
2. $S_{2..12}$ subscribe; all reach R (beacons steer near-misses). At 21 directs, R promotes S_2 to a child relay with a batch of 8 — the tree widens.
3. P publishes: envelope signed with P 's author key; PUB routes toward T (hint-assisted); R verifies, stamps (`publishTs, seq=1`),

caches, fans to directs and the child (which re-fans), replicates eagerly to its 2-member cohort. P , being subscribed, receives its own message — observation confirms; retries stop.

4. P kills the message: signed kill routes to R ; authorship matches; tombstone recorded (**seq**=2), del marker fans down, cohort updated eagerly. A later **since**: 'all' joiner replays survivors only.
5. R leaves: drain (nothing pending), handoff of T 's cache+tombstones to heir H (**from**: R), notify, silence. H purges R 's beacon, keeps the claim, beacons; renewals re-seat everyone under H within one fast cycle. Total order continues above the inherited **lastTs**.

Axona Architecture v4.21.0 — 2026-07-13. Versioned to the kernel release deployed on testnet that it describes; re-version on the release that next changes protocol behavior. Companions: Root-Management-v4.20.1, INVARIANTS.md, Kernel-Refactor-Analysis-v0.1, Soak-Framework-Overview-v4.21.0.